# EduTrack: Student Project Management System

A MINOR PROJECT REPORT SUBMITTED TO

**THE NATIONAL INSTITUTE OF ENGINEERING, MYSURU**
**(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)**




in partial fulfillment for the award of degree of

**Bachelor of Engineering**
**in**
**Computer Science and Engineering**

Submitted By


| | |
|---|---|
| **Khushal** | **4NI24CS097** |
| **Mayank Dangi** | **4NI24CS115** |
| **Mohak Kharbanda** | **4NI24CS121** |
| **Naresh** | **4NI25CS412** |


**Under the guidance of**


Poornima N
Project Instructor
Department of CS&E
NIE, Mysuru






**Department of Computer Science & Engineering**
THE NATIONAL INSTITUTE OF ENGINEERING
(Autonomous Institution)
Mysuru - 570 008

2026-27

**THE NATIONAL INSTITUTE OF ENGINEERING**

(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)

**Department of Computer Science & Engineering**

# *CERTIFICATE*

This is to certify that the project work entitled "**EduTrack: Student Project Management System**" is a bonafide work carried out by **Khushal (4NI24CS097), Mayank Dangi (4NI24CS115), Mohak Kharbanda (4NI24CS121) and Naresh (4NI25CS412)** in partial fulfillment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering**, of Visvesvaraya Technological University, Belagavi, during the year **2026-27**. It is certified that all corrections / suggestions indicated during internal assessment have been incorporated and the corrected copy has been deposited in the department library. This project report has been approved in partial fulfillment for the award of the said degree as per academic regulations of The National Institute of Engineering (Autonomous Institution).

| | |
|---|---|
| **Poornima N**<br>**Project Instructor**<br>Dept. of CSE<br>NIE, Mysuru | **Dr. Anitha R**<br>**Professor and Head**<br>Dept. of CS&E<br>NIE, Mysuru |

| **Name of the Examiners** | **Signature with Date** |
|---|---|
| **1.** _____ | _____ |
| **2.** _____ | _____ |

# ABSTRACT

Final-year and minor projects in engineering colleges are usually managed through a mix of WhatsApp groups, e-mails, pen drives and spreadsheets. Because of this, it is hard for students to divide the work, for guides to know how far each team has reached, and for the department to keep a clear record of proposals, feedback and marks.

This report presents **EduTrack**, a web-based Student Project Management System that brings the whole life of an academic project into one place. Students form teams using a join code, write a project proposal, break the work into tasks with deadlines, assign tasks to teammates and upload their files. Faculty guides approve or reject proposals, give feedback, open the files uploaded by students, upload a corrected version with a comment, follow each team's progress and award marks. Administrators manage students, faculty, departments and projects, and can see every team in the institution ranked by progress.

Progress is calculated automatically from the tasks that are marked as done and is shown everywhere as a simple bar, for example “60% complete”. The system is written in JavaScript, with React on the frontend, Node.js and Express on the backend, and MongoDB as the database. Security is handled with JSON Web Tokens, bcrypt password hashing and permission checks on the server for every request. Every uploaded file keeps its full version history and can be stored on the server's disk, on Cloudinary or on Amazon S3.

The system was tested with 55 automated API tests and 8 user-interface tests, all of which pass, along with manual testing of every role in the browser. It is packaged with Docker and Render configuration files so that it can be deployed to a cloud host.

# ACKNOWLEDGMENT

We sincerely owe our gratitude to all the persons who helped and guided us in completing this minor project.

We are thankful to **Dr. B. S. Nagendra Parashar**, Principal, NIE, Mysuru, for all the support rendered to us.

We thank **Dr. Anitha R**, Professor and Head, Department of Computer Science and Engineering, for the constant support and encouragement throughout the tenure of the minor project work.

We would like to sincerely thank our guide **Poornima N**, Project Instructor, Department of Computer Science and Engineering, for providing relevant information, valuable guidance and encouragement to complete this minor project.

We also thank our friends and families for their patience and support while we worked on this project.

| | |
|---|---|
| Khushal | 4NI24CS097 |
| Mayank Dangi | 4NI24CS115 |
| Mohak Kharbanda | 4NI24CS121 |
| Naresh | 4NI25CS412 |

# TABLE OF CONTENTS


| CONTENTS | PAGE NO. |
|---|---|
| Abstract | i |
| Acknowledgment | ii |
| Table of Contents | iii |
| List of Figures | iv |
| **1. Introduction** | **1** |
| 1.1 Background | 1 |
| 1.2 Purpose | 1 |
| 1.3 Objectives | 2 |
| **2. Literature Review** | **3** |
| **3. Existing System and Proposed System** | **6** |
| 3.1 Existing System | 6 |
| 3.2 Proposed System | 7 |
| **4. System Requirements** | **9** |
| 4.1 Hardware Requirements | 9 |
| 4.2 Software Requirements | 9 |
| **5. System Design** | **11** |
| 5.1 System Architecture | 11 |
| 5.2 Use Case Diagram | 15 |
| **6. Implementation** | **17** |
| 6.1 Pseudocode | 20 |
| **7. Testing** | **23** |
| **8. Screenshots** | **26** |
| **Future Enhancement and Conclusion** | **35** |
| **References** | **36** |

# LIST OF FIGURES


| FIG. NO. | DESCRIPTION | PAGE NO. |
|---|---|---|
| 5.1 | System architecture of EduTrack | 11 |
| 5.2 | Database design (entity–relationship diagram) | 13 |
| 5.3 | Life cycle of a project in EduTrack | 14 |
| 5.4 | Use case diagram of EduTrack | 16 |
| 8.1 | Home page of EduTrack | 26 |
| 8.2 | Sign-in page with demo accounts | 27 |
| 8.3 | Student dashboard showing project progress, tasks and deadlines | 27 |
| 8.4 | Project workspace: overview with the project journey | 28 |
| 8.5 | Task board with To do, In progress, In review and Done columns | 28 |
| 8.6 | Files tab with the upload area and version labels | 29 |
| 8.7 | Team page with members and the join code | 29 |
| 8.8 | Faculty dashboard with progress bars of guided teams | 30 |
| 8.9 | Faculty view of their teams sorted by progress | 30 |
| 8.10 | File details with review status, version history and comments | 31 |
| 8.11 | Proposal waiting for the guide's approval | 31 |
| 8.12 | Administrator dashboard | 32 |
| 8.13 | All teams in the institution ranked by progress | 32 |
| 8.14 | User management for the administrator | 33 |
| 8.15 | Audit log of important actions | 33 |
| 8.16 | Student dashboard on a phone in the dark theme | 34 |

# Chapter 1

# INTRODUCTION

## 1.1 Background

Every engineering student completes several team projects during the course, such as a mini project, a minor project and a major project. A project normally passes through many stages: forming a team, choosing a topic, getting the proposal approved by a guide, dividing the work, sharing documents, regular reviews, final submission and evaluation.

In most colleges these stages are handled with general tools that were never designed for this purpose. Teams talk on WhatsApp, send documents by e-mail or pen drive, keep task lists in notebooks or spreadsheets, and meet the guide in person to show their progress. Each faculty member guides several teams at the same time and has to remember where each team stands. The department keeps proposals, review remarks and marks in paper files or separate spreadsheets.

This scattered way of working causes common problems. Files get lost or overwritten, nobody is sure which copy of the report is the latest, guides have no quick way to see which team is falling behind, and students do not get written feedback in one place. At evaluation time, collecting the final documents and marks takes a lot of extra effort.

Web applications are now used for almost every kind of team work, and students and teachers are comfortable using them on laptops and phones. This makes it practical to build a dedicated system that manages an academic project from the first idea to the final marks.

## 1.2 Purpose

The purpose of this project is to build **EduTrack**, a single web platform where students, faculty and administrators manage the complete life of an academic project. Instead of messages spread across many apps, every proposal, task, file, comment, review and mark is stored in one place and is visible only to the people who should see it.

EduTrack is meant to make the following things easy:

- **For students:** organising the team, planning the work as tasks with deadlines, sharing files and always knowing how much of the project is done.
- **For faculty:** reviewing proposals, reading and correcting the team's files, giving written feedback and awarding marks, while seeing the progress of every team they guide.
- **For administrators:** managing accounts, departments and projects, and seeing the progress of all teams in the institution at a glance.

A second purpose was to build the system the way real software is built: with automated tests, security rules that are checked on the server, and configuration files that allow the application to be deployed to the cloud.

## 1.3 Objectives

The objectives of the project, as stated in our synopsis, are:

1. **Centralisation:** build one web platform where students, faculty and the administrator handle the whole project life cycle, replacing scattered e-mails, spreadsheets and in-person follow-ups.
2. **Team and project management:** let students create a team, add members with a join code or by e-mail, and write a clear project proposal that everyone involved can read.
3. **Task tracking:** let teams create tasks, assign them to members with deadlines, and calculate progress automatically from completed tasks, with overdue tasks clearly highlighted.
4. **File and submission management:** provide secure file upload with type and size limits, keep every version of a file along with comments, and support a final submission step.
5. **Faculty oversight:** give guides tools to approve or reject proposals, give feedback, edit and re-upload files with a comment, track progress and award marks.
6. **Role-based administration:** give administrators a dashboard to manage students, faculty, departments and projects, view all teams by progress and read a log of important actions.
7. **Deployment readiness:** build a complete frontend, backend and database that is tested and packaged with the files needed to deploy it to a cloud host.

# Chapter 2

# LITERATURE REVIEW

Before designing EduTrack we studied the tools that colleges and teams already use, and the standard techniques for building secure web applications. This chapter summarises what we learned and how it shaped our design.

### Learning management systems

Moodle [1] is an open-source learning management system used by many universities. It supports courses, assignments, quizzes, grading and student groups. Google Classroom [2] lets teachers create classes, share material, and collect and grade assignments. These tools work well for course work given to a whole class. However, they are organised around courses and assignments, not around a team project that runs for a full semester. They do not offer a proposal approval step, a task list for each team member, or a progress figure that a guide can compare across teams.

### General project management tools

Trello [3] organises work as cards on boards, and Jira [4] is widely used by software teams to track issues and sprints. Both are very good at task tracking. However, they are built for companies: there is no idea of a faculty guide, of a proposal that must be approved before work starts, of marks or of departments. Each team would have to set up its own board, and a guide would have to join many separate boards to follow their teams.

### Code-centred classroom tools

GitHub Classroom [5] helps teachers hand out programming assignments as code repositories and collect them. It suits source code, but not documents such as reports and presentations, and it does not track the tasks or the progress of a team.

### Task boards and the Kanban method

The Kanban method described by Anderson [6] shows work as cards that move through columns such as “To do”, “In progress” and “Done”. Making work visible in this way helps a team see

what is finished and what is stuck. EduTrack uses a Kanban-style board with four columns (To do, In progress, In review and Done) and calculates progress as the share of tasks that have reached the Done column.

**Role-based access control**

Sandhu et al. [7] describe role-based access control (RBAC), where permissions are given to roles and each user is given a role. This fits a college well, because students, faculty and administrators have clearly different powers. EduTrack combines roles with ownership rules. For example, a faculty member can review a proposal only if they are the guide assigned to that project, and a student can see only their own team's project.

**Web application architecture**

Fielding [8] defined the REST style, in which a server offers resources through standard HTTP methods such as GET and POST. EduTrack's backend is a REST API built with Express [9] on Node.js, and its frontend is a single-page application built with React [10]. MongoDB [11] stores data as flexible documents, which suits records such as a project that holds lists of feedback, reviews and task statistics. Mongoose [15] is used to define the structure of these documents and their indexes.

**Authentication and password storage**

JSON Web Tokens (JWT), defined in RFC 7519 [12], are signed tokens that let a server check who is making a request. Passwords must never be stored as plain text; Provos and Mazières [13] designed bcrypt, a slow and salted hashing method that makes a stolen password database very hard to crack. EduTrack uses both, and keeps the long-lived login token in an HTTP-only cookie as described in RFC 6265 [18], so that scripts running in the page cannot read it.

**Web application security**

The OWASP Top 10 [14] lists the most serious security risks for web applications. “Broken access control” is ranked first in the 2021 list. This means that hiding a button in the user interface is not enough; every request must be checked on the server. EduTrack follows this rule: every protected API endpoint checks the user's role and their relation to the project before doing anything. All input is checked against validation rules written with Zod [17].

**Summary of the review**

Table 2.1 compares the existing tools with EduTrack. Each tool covers part of the problem, but none of them covers the full life of an academic team project with separate roles for students, guides and administrators.

**Table 2.1: Comparison of existing tools with EduTrack**

| Feature | LMS (Moodle, Classroom) | PM tools (Trello, Jira) | EduTrack |
|---|---|---|---|
| Team formation | Groups set up by the teacher | Workspace invitations | Students create or join a team with a code |
| Proposal approval | Not built in | Not built in | Approved or rejected by the assigned guide |
| Tasks with deadlines | Not built in | Yes | Yes, assigned to team members |
| Progress percentage | Not built in | Through reports or add-ons | Automatic, shown as a progress bar |
| File versions with comments | Submission per assignment | File attachments | Full version history with notes and comments |
| Marks and evaluation | Yes | No | Yes, by the guide |
| View of all teams | Per course | Per board | All teams ranked by progress |

# Chapter 3

# EXISTING SYSTEM AND PROPOSED SYSTEM

## 3.1 Existing System

In the existing system, academic projects are managed manually with the help of general communication tools. A typical project goes through the following steps:

1. Students form teams informally and tell the guide or the project coordinator.
2. The proposal is written as a document and shared by e-mail or as a printout. Approval is given in person or by signature.
3. Work is divided in WhatsApp chats, and deadlines are remembered or noted in personal notebooks.
4. Files are shared through e-mail, Google Drive links and pen drives, which leads to many copies such as “report_final_v2_new.docx”.
5. The guide checks progress in weekly meetings by asking each team what it has done.
6. The final report is submitted as a printout or a CD, and marks are entered in a spreadsheet.

**Drawbacks of the existing system**

- **No single source of truth:** information is spread across chats, mailboxes and drives.
- **No visibility of progress:** a guide cannot see at a glance which team is on track and which is falling behind.
- **Feedback gets lost:** remarks given verbally or inside long chat threads are easily forgotten.
- **Version confusion:** nobody is sure which copy of a document is the latest one, and corrections are overwritten.
- **No record of individual contribution:** it is hard to see who did which part of the work.
- **Extra effort at evaluation:** collecting final files and marks from every team takes a lot of time for faculty and the department.
- **Weak access control:** shared links can often be opened by anyone who has them.

## 3.2 Proposed System

The proposed system, **EduTrack**, is a web application that manages the complete life of an academic project. Every user signs in with their own account, and what they can see and do depends on their role. All information is stored in one database, and progress is calculated automatically.

**Features for students**

- Create a team and share its six-character join code, or join a team with a code. A team has up to five members by default, and a student can belong to only one team.
- Add teammates by e-mail; the team leader can remove members.
- Write a project proposal with a title, description, objectives, technologies, faculty guide and deadline, and either save it as a draft or send it for review.
- Plan the work on a task board: create tasks, assign them to teammates, set priorities and due dates, and drag tasks between columns. Overdue tasks are highlighted in red.
- Upload project files and, at the end, final deliverables, and submit the project for evaluation.
- See a personal dashboard with the project's progress ring, their own tasks, upcoming deadlines and each teammate's contribution.

**Features for faculty**

- See only the teams they guide, with a progress bar for each team on their dashboard.
- Approve or reject proposals. A rejection must include feedback so the team knows what to change.
- Open the files uploaded by students, preview them in the browser, mark them as accepted or as needing changes, and add comments.
- Edit a student's file and upload the corrected version with a required comment. It becomes the next version, and all older versions are kept.
- Return a submitted project for changes, or evaluate it by awarding marks out of 100 with remarks.

**Features for administrators**

- Create student and faculty accounts, deactivate or reactivate them, and generate one-time password reset links.
- Manage departments and projects, reassign a project's faculty guide, and delete projects when needed.
- See all teams in the institution ranked by progress, with search, filters and sorting, and see the workload of each faculty member.
- Read the audit log, which records who did what and when.

**Advantages of the proposed system**

- All project information is in one place and is always up to date.
- Progress is visible to students, guides and administrators as a simple percentage.
- Every version of every file is kept, together with the comments explaining each change.
- Permissions are checked on the server, so users can only see and change what their role allows.
- The application works on computers and phones, in light and dark themes, and refreshes its data automatically.

# Chapter 4

# SYSTEM REQUIREMENTS

This chapter lists the hardware and software needed to develop, run and use EduTrack.

## 4.1 Hardware Requirements

**Table 4.1: Hardware requirements**

| Purpose | Minimum requirement |
|---|---|
| Development computer | 8 GB RAM, modern multi-core processor (Intel Core i5, AMD Ryzen 5 or Apple M-series), 10 GB of free disk space, running Windows, macOS or Linux |
| Application server | Cloud host such as Render, AWS or DigitalOcean with at least 1 virtual CPU and 1 GB RAM |
| Database server | MongoDB Atlas cluster (the free tier is enough to start) or any server running MongoDB 7 |
| File storage | Cloudinary or Amazon S3 account; the server's own disk can be used for small installations |
| End user device | Any PC, laptop, tablet or smartphone with an up-to-date web browser and an internet connection |

## 4.2 Software Requirements

**Table 4.2: Software requirements**

| Layer | Software | Version | Used for |
|---|---|---|---|
| Runtime | Node.js | 22 LTS | Running the backend and the build tools |
| Language | JavaScript (ES modules) | ES2022 | All frontend and backend code |
| Frontend | React | 18.3 | Building the user interface |
| | Vite | 6 | Development server and production build |
| | Tailwind CSS | 3.4 | Styling and light/dark themes |
| | Framer Motion | 11 | Animations |
| | TanStack Query | 5 | Loading server data and auto-refresh |

| Layer | Software | Version | Used for |
|---|---|---|---|
| | React Router | 7 | Page navigation |
| | Bricolage Grotesque, Geist | — | Fonts (Google Fonts) |
| Backend | Express | 4.21 | REST API |
| | Mongoose | 8 | Database models and indexes |
| | Zod | 4 | Validating all input |
| | jsonwebtoken, bcryptjs | 9, 2.4 | Login tokens and password hashing |
| | Helmet, express-rate-limit | 8, 7 | Security headers and rate limiting |
| | Multer | 2 | Receiving file uploads |
| | Nodemailer | 10 | Password reset e-mails (optional) |
| Database | MongoDB | 7 | Storing all application data |
| File storage | Cloudinary / Amazon S3 | — | Storing uploaded files in the cloud |
| Testing | Jest, Supertest | 29, 7 | Automated API tests |
| | mongodb-memory-server | 10 | Real MongoDB server for tests |
| | Vitest, Testing Library | 4, 16 | User-interface tests |
| Tools | Git and GitHub | — | Version control and CI |
| | Visual Studio Code | — | Code editor |
| | Docker, Render | — | Packaging and cloud deployment |
| Client | Chrome, Edge, Firefox, Safari | Latest | Using the application |

# Chapter 5

# SYSTEM DESIGN

This chapter describes how EduTrack is put together: the main parts of the system, how data is stored, how a project moves from one stage to the next, and what each type of user can do.

## 5.1 System Architecture

EduTrack follows a three-tier architecture, shown in Figure 5.1. The **frontend** is a React single-page application that runs in the user's browser. The **backend** is a REST API written with Node.js and Express. The **storage tier** holds the MongoDB database, the uploaded files and, optionally, an e-mail server for password reset links. A small **shared package** contains the constants, validation rules and progress calculation used by both the frontend and the backend, so the two sides always agree.

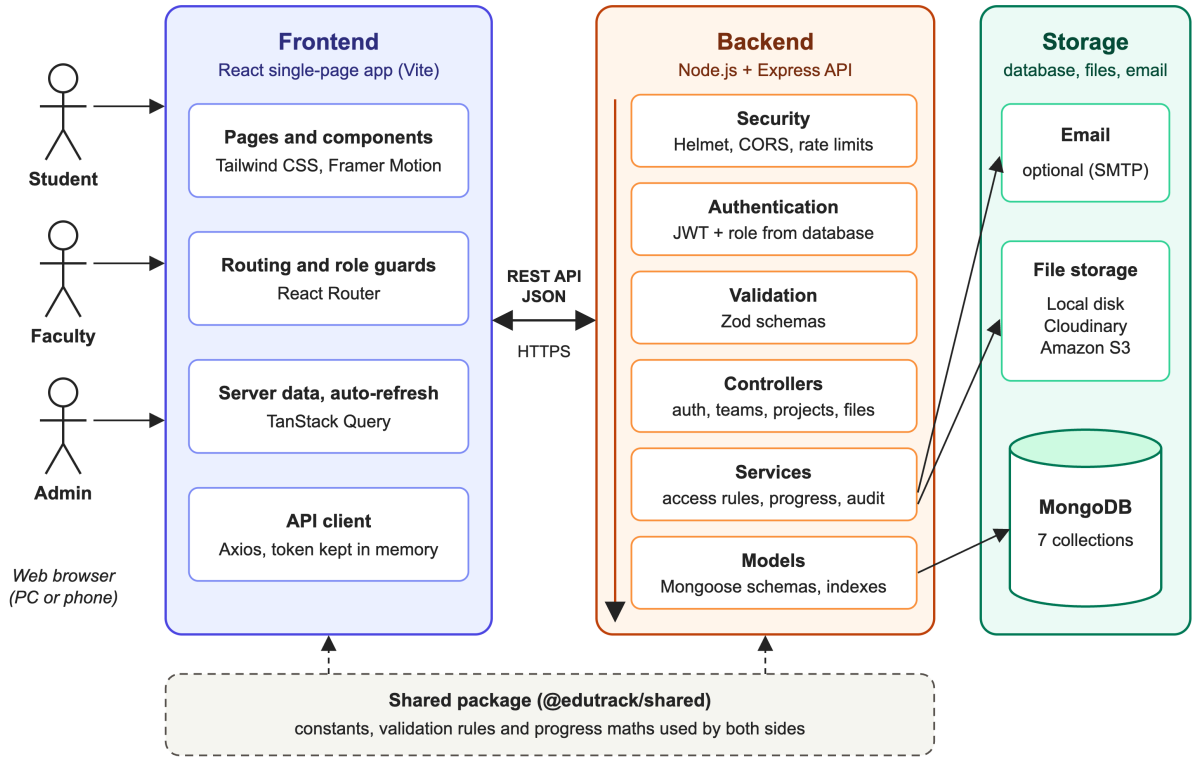



**Fig. 5.1: System architecture of EduTrack**

When a user performs an action, the request passes through the backend in a fixed order:

1. The browser sends the request to the API with a short-lived access token.
2. Security middleware adds safe HTTP headers (Helmet), allows only the college's own website to call the API (CORS) and limits the number of requests from one address (rate limiting).
3. The authentication step checks the token, loads the user from the database and confirms that the account is still active. The role is always read from the database, so a role change takes effect immediately.
4. The request body and query are checked against Zod validation rules. Invalid input is refused with a clear message.
5. The controller loads the project, if any, and checks that the user is allowed to see it: a team member, the assigned guide or an administrator.
6. Services do the actual work, such as recalculating progress or saving a file, and record the action in the audit log.
7. The API replies with a JSON response that the frontend shows to the user.

**Database design**

The data is stored in seven MongoDB collections. Figure 5.2 shows the collections, their main fields and how they are related, and Table 5.1 explains what each one holds.

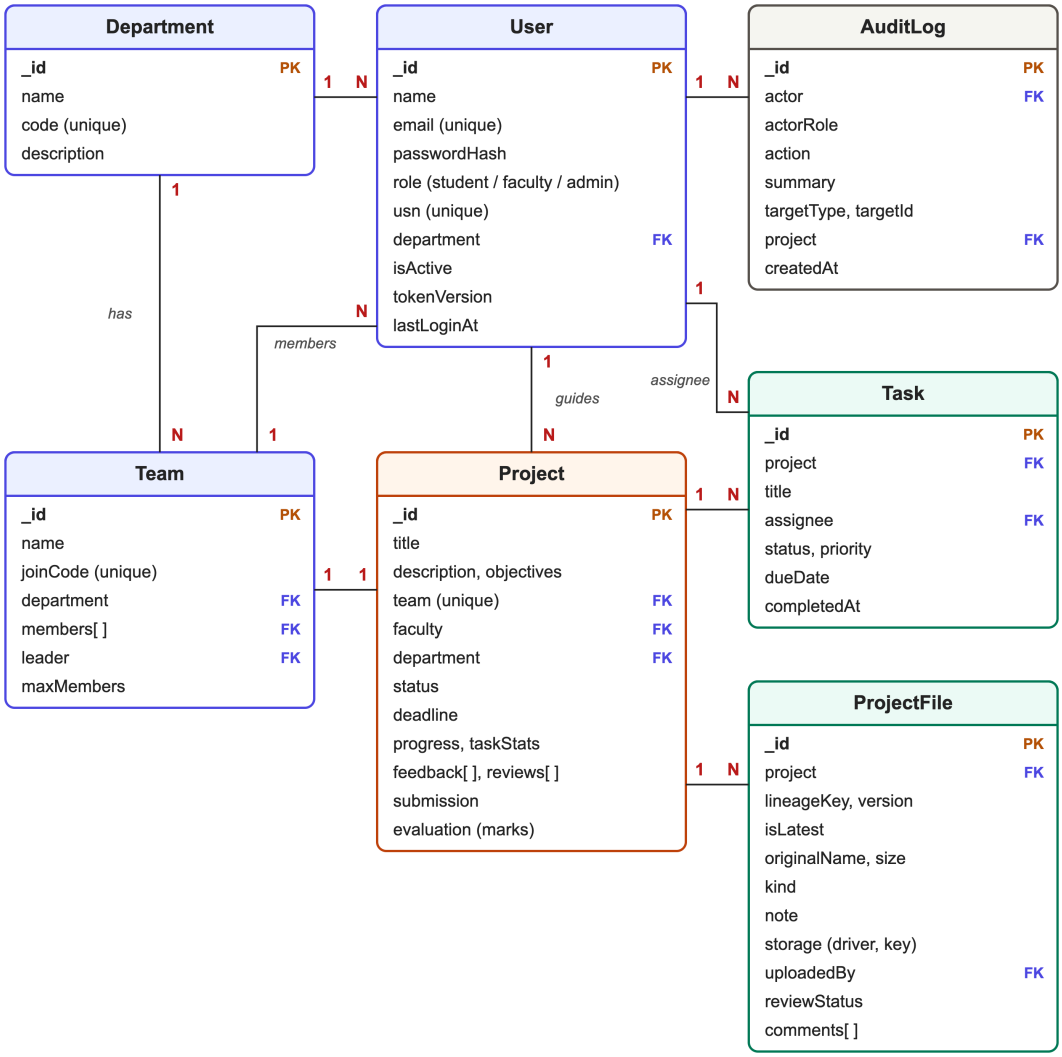



**Fig. 5.2: Database design (entity–relationship diagram)**

**Table 5.1: Collections in the database**

| Collection | What it stores | Important rules |
|---|---|---|
| users | Accounts for students, faculty and administrators | Unique e-mail; unique USN for students; password stored only as a bcrypt hash |
| departments | Departments of the institution | Unique department code |
| teams | Team name, join code, members and leader | Unique join code; a student can be a member of only one team (unique index on members) |
| projects | Proposal, guide, status, deadline, feedback, reviews, submission and marks | One project per team; progress and task counts are stored for fast sorting |

| Collection | What it stores | Important rules |
|---|---|---|
| tasks | Tasks with assignee, status, priority and due date | Assignee must be a member of the project's team |
| projectfiles | Every uploaded file and every version of it, with notes, review status and comments | Project, file lineage and version number together are unique; isLatest marks the current version |
| auditlogs | Who did what and when | Used for the activity feed and the administrator's audit log |

### Project life cycle

Every project has a status that tells everyone which stage it has reached. Figure 5.3 shows the possible statuses and what moves a project from one to the next.

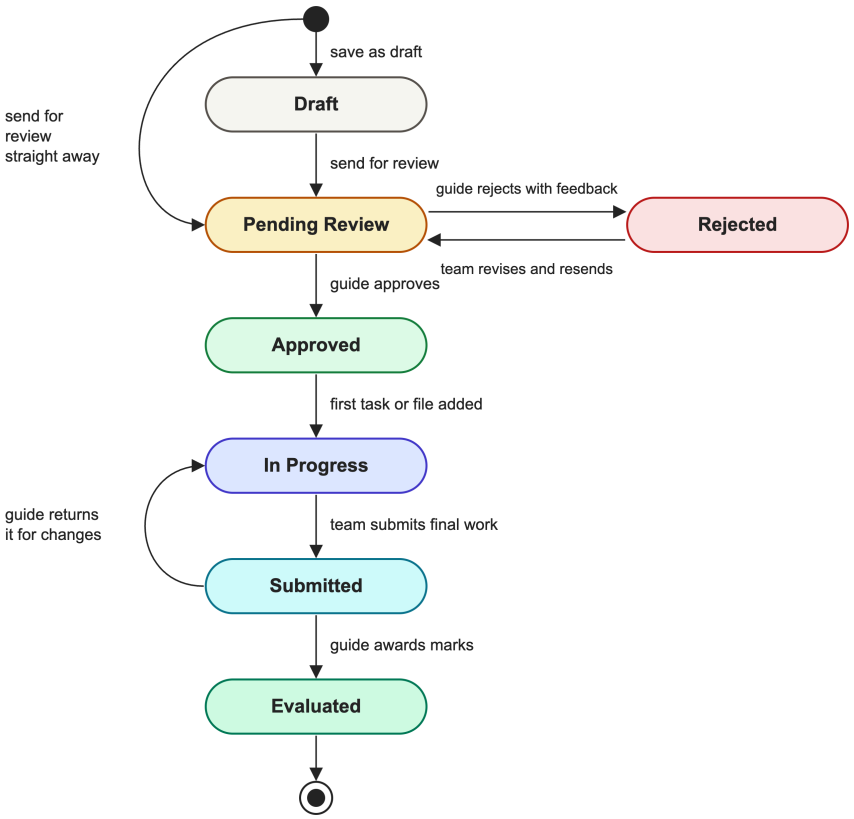



**Fig. 5.3: Life cycle of a project in EduTrack**

- **Draft:** the team is still writing the proposal. The guide cannot see it yet.
- **Pending Review:** the proposal has been sent to the guide.

- **Rejected:** the guide asked for changes and wrote feedback. The team revises the proposal and sends it again.
- **Approved:** the guide accepted the proposal. The team can now add tasks and files.
- **In Progress:** the team has started work. This status is set automatically when the first task or file is added.
- **Submitted:** the team uploaded its final deliverables and submitted the project. Tasks and team membership are now locked.
- **Evaluated:** the guide awarded marks. The guide can also return a submitted project to In Progress if more work is needed.

## 5.2 Use Case Diagram

Figure 5.4 shows the use case diagram. There are three main actors: **Student**, **Faculty** and **Admin**. All three are kinds of **User**, so they share the use cases at the bottom of the diagram: signing in and managing their account, viewing progress bars, and commenting on files and feedback. The use cases on the left belong to students, those at the top right to faculty guides, and those at the bottom right to administrators.

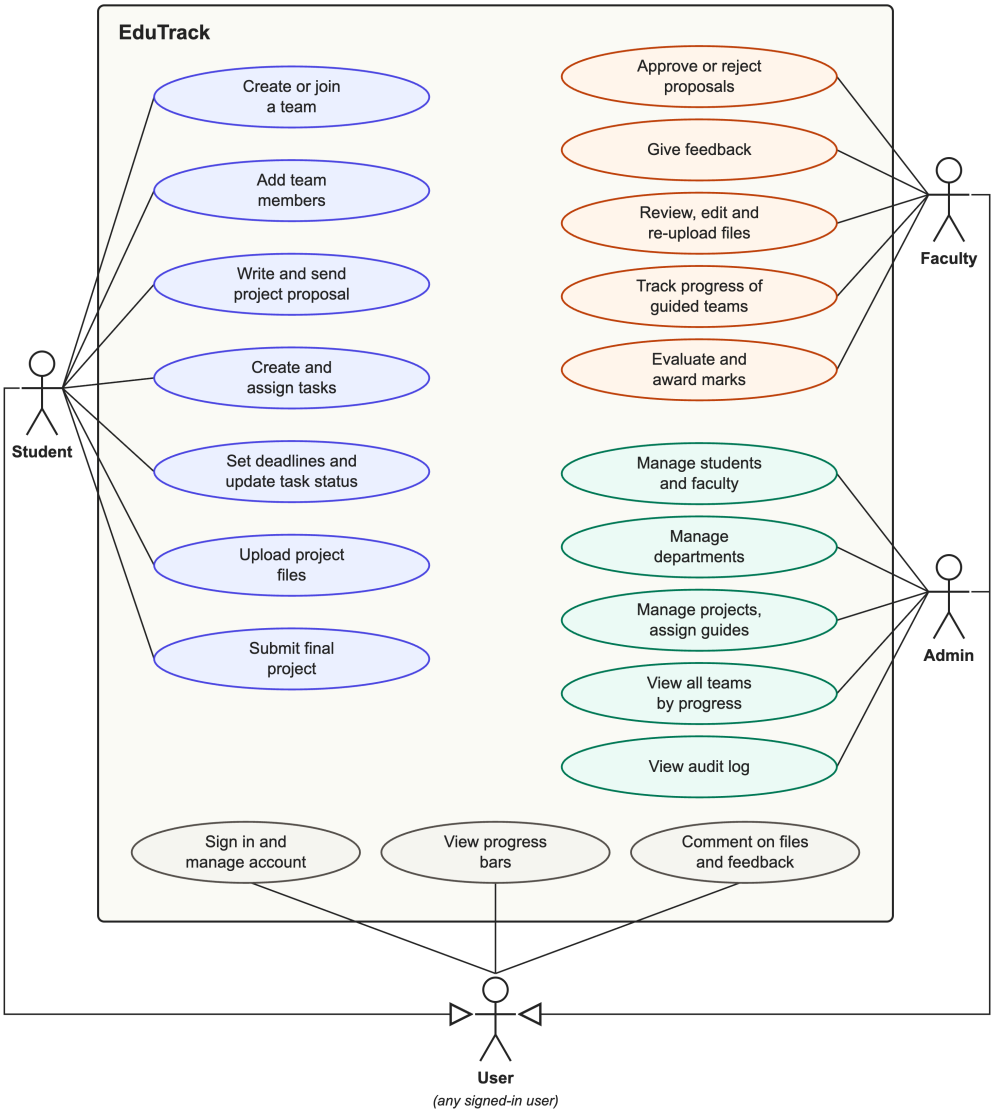



**Fig. 5.4: Use case diagram of EduTrack**

# Chapter 6

# IMPLEMENTATION

EduTrack is written entirely in JavaScript using ES modules. The frontend pages and components are React JSX files, and the backend runs directly on Node.js with no compile step. The code is kept in a single repository divided into three packages using npm workspaces: a shared package, the backend and the frontend. Table 6.1 shows the main folders.

**Table 6.1: Project folder structure**

| Folder | Contents |
|---|---|
| spms-shared/src | Shared constants, Zod validation rules and progress functions |
| spms-backend/src/config | Environment settings and the database connection |
| spms-backend/src/models | Mongoose models for the seven collections |
| spms-backend/src/middleware | Authentication, role checks, validation, uploads, rate limits and error handling |
| spms-backend/src/controllers | Request handlers for each feature |
| spms-backend/src/services | Access rules, progress, audit log, file storage drivers and e-mail |
| spms-backend/src/routes | All API routes in one place |
| spms-backend/scripts | Demo data, first administrator account and a local database |
| spms-backend/tests | Automated API tests |
| spms-frontend/src/pages | Pages for students, faculty and administrators |
| spms-frontend/src/features | Project workspace parts: task board, files, feedback and activity |
| spms-frontend/src/components | Reusable parts such as buttons, progress bars and dialogs |
| Root folder | Dockerfile, docker-compose.yml, render.yaml and the GitHub Actions workflow |

### Authentication and sessions

When a user signs in, the server checks the password against the bcrypt hash and returns two tokens. The access token is valid for 15 minutes and is kept only in the browser's memory. The refresh token is valid for 7 days and is stored in an HTTP-only cookie that page scripts cannot

read. When the access token expires, the frontend quietly asks for a new one; the cookie is replaced each time. Changing the password or choosing “sign out of all devices” increases a version number stored with the user, which makes every older token invalid. Public sign-up can only create student accounts; faculty accounts are created by the administrator, and the first administrator is created from environment variables or with a command. If e-mail is configured, users can reset a forgotten password through an e-mailed link; otherwise the administrator can generate a one-time reset link. Reset tokens are stored only as SHA-256 hashes and expire after a short time.

**Teams, projects and reviews**

A team is created with a random six-character join code. The database itself prevents a student from being in two teams, through a unique index on the members field. A project belongs to exactly one team and has one faculty guide. Only the assigned guide can approve or reject the proposal, and only after it has been sent for review. Each review and each piece of feedback is stored with the project, so the full discussion is always available.

**Tasks and progress**

Tasks can be added only after the proposal is approved, and can be assigned only to members of the same team. Whenever a task is created, updated or deleted, the server recalculates the project's task counts and its progress percentage and saves them on the project. Storing these values lets dashboards sort hundreds of teams by progress quickly. Whether a task is overdue depends on the current time, so overdue counts are always calculated fresh from the tasks collection.

**Files and versions**

Uploaded files are received with Multer, checked against a list of allowed types (PDF, Word, PowerPoint, Excel, CSV, text, Markdown, ZIP and common image formats) and limited to 25 MB. They are then saved through a storage driver that can use the server's disk, Cloudinary or Amazon S3; the choice is made with an environment variable, and storage keys never leave the server. Each new version of a file is saved as a separate record linked to the earlier versions, so nothing is ever overwritten. When a guide or administrator uploads a corrected version, a

comment explaining the change is required. Files are downloaded through signed links that expire after five minutes, and PDF, image and text files can be previewed inside the application.

**Dashboards and the progress boards**

Each role has its own dashboard. Students see their project's progress ring, their tasks and upcoming deadlines. Faculty see a progress bar for every team they guide, the proposals waiting for review and the submissions waiting for marks. Administrators see institution-wide numbers and a full list of teams ranked by progress, with search, filters, sorting and pages. The frontend uses TanStack Query [16] to refresh this data automatically every 15 to 45 seconds, and each page shows an "Auto-refresh" label so users know how current the data is.

**Security measures**

- Strict security headers, including a Content Security Policy that blocks inline scripts, set with Helmet.
- Only the configured website address can call the API (CORS allow-list).
- Rate limits on sign-in, password reset and the API in general, to slow down password guessing.
- Passwords hashed with bcrypt using 12 rounds.
- Every input validated with Zod, and every database ID checked before it is used.
- Error messages that never reveal internal details, and secrets that are read only from environment variables.

**User interface**

The interface uses the Bricolage Grotesque font for headings, Geist for text and Geist Mono for numbers. Colours are defined once as variables, which gives the application matching light and dark themes. Framer Motion adds smooth page transitions, animated progress numbers and a drag-and-drop task board, and all animation is reduced automatically for users who turn on "reduce motion" in their device settings. Every page works on phone screens, progress bars are announced correctly by screen readers, and pages show clear loading, empty and error states.

**Deployment**

In production, the Express server also serves the built frontend, so the whole application runs as one web service. The repository includes a render.yaml blueprint for Render, a multi-stage Dockerfile, a docker-compose.yml file that starts the application together with MongoDB, a health check at /api/health and .env.example files that list every setting. A GitHub Actions workflow builds the project and runs all tests on every push. The application has been prepared for deployment in this way; going live requires the institution's own MongoDB Atlas and Cloudinary (or Amazon S3) accounts.

**Table 6.2: Main API endpoints**

| Area | Endpoints | Who can use them |
|---|---|---|
| Sign-in | POST /api/auth/register, /login, /refresh, /logout, /forgot-password, /reset-password | Everyone (sign-up creates students only) |
| Account | GET and PATCH /api/auth/me, POST /api/auth/change-password, /logout-all | Any signed-in user |
| Teams | POST /api/teams, /api/teams/join, /api/teams/:id/members; GET /api/teams/mine | Students |
| Team progress | GET /api/teams/progress | Faculty (own teams) and admin (all) |
| Projects | GET and POST /api/projects; GET and PATCH /api/projects/:id | Limited to what the viewer may see |
| Reviews and marks | POST /api/projects/:id/review, /feedback, /submit, /return, /evaluate | Guide, team or admin, as allowed |
| Tasks | GET and POST /api/projects/:id/tasks; PATCH and DELETE …/tasks/:taskId | Team members |
| Files | GET and POST /api/projects/:id/files; …/comments, …/review, …/url | Team, guide and admin |
| Administration | /api/users, /api/departments, /api/admin/audit-logs | Admin |

## 6.1 Pseudocode

The most important parts of the system are described below in pseudocode.

**Algorithm 1: Signing in and keeping the session alive**

```
LOGIN(email, password)
    user ← find user by email
```

```
    // the same message is used for a wrong e-mail and a wrong password
    IF user not found OR password does not match user.passwordHash THEN
        RETURN error "Invalid email or password"
    IF user is deactivated THEN RETURN error "Account disabled"
    access  ← sign JWT {user id, tokenVersion}, valid 15 minutes
    refresh ← sign JWT {user id, tokenVersion}, valid 7 days
    set refresh as an HTTP-only cookie
    RETURN access token and user details

REFRESH(cookie)
    IF request lacks the "X-Requested-With" header THEN RETURN error
    token ← verify cookie; user ← find user by token.id
    IF user inactive OR token.tokenVersion ≠ user.tokenVersion THEN
        RETURN error
    issue new access token and replace the refresh cookie
```

### Algorithm 2: Checking whether a user may open a project

```
LOAD_PROJECT_CONTEXT(projectId, user)
    project ← find project by projectId;  team ← find team of project
    isAdmin    ← user.role = "admin"
    isMember   ← user.role = "student" AND user is in team.members
    isGuide    ← user.role = "faculty" AND project.faculty = user
                 AND project.status ≠ "draft"
    IF NOT (isAdmin OR isMember OR isGuide) THEN
        RETURN error 403 "You do not have access to this project"
    RETURN {project, team, isAdmin, isMember, isGuide}

Every project action first calls LOAD_PROJECT_CONTEXT, then checks the
specific permission, e.g. canReview ← isGuide AND status = "pending_review"
```

### Algorithm 3: Calculating project progress

```
RECOMPUTE_PROGRESS(projectId)           // runs after every task change
    tasks ← all tasks of the project
    stats ← {total, todo, inProgress, review, done, overdue all = 0}
    FOR EACH task IN tasks
        stats.total ← stats.total + 1
        increase the counter for task.status
        IF task.dueDate < now AND task.status ≠ "done" THEN
            stats.overdue ← stats.overdue + 1
    IF stats.total = 0 THEN progress ← 0
    ELSE progress ← ROUND(stats.done / stats.total × 100)
    save stats and progress on the project
```

### Algorithm 4: Uploading a file or a new version

```
UPLOAD_FILE(projectId, user, file, lineageKey, note)
    ctx ← LOAD_PROJECT_CONTEXT(projectId, user)
    check file type against the allowed list and size ≤ 25 MB
    IF lineageKey is given THEN                    // a new version
        previous ← latest file with this lineageKey
        version  ← previous.version + 1
        IF user is not a team member AND length(note) < 3 THEN
            RETURN error "Add a comment explaining what you changed"
    ELSE
        lineageKey ← new random id;  version ← 1
    IF user may not upload at this project stage THEN
        RETURN error 403
    stored ← storage driver saves the file (disk, Cloudinary or S3)
    create file record {lineageKey, version, isLatest: true, note}
    IF previous exists THEN previous.isLatest ← false
    IF project.status = "approved" THEN project.status ← "in_progress"
    write audit log entry
```

### Algorithm 5: Reviewing a proposal

```
REVIEW_PROPOSAL(projectId, faculty, decision, message)
    ctx ← LOAD_PROJECT_CONTEXT(projectId, faculty)
    IF NOT ctx.isGuide THEN RETURN error "Only the guide can review"
    IF project.status ≠ "pending_review" THEN RETURN error
    IF decision = "rejected" AND message is empty THEN RETURN error
    project.status ← decision                // approved or rejected
    add {decision, message, faculty, time} to project.reviews
    IF message given THEN add it to project.feedback
    save project and write audit log entry
```

### Algorithm 6: Building the all-teams progress board

```
TEAMS_PROGRESS(user, search, filters, sort, page)
    IF user.role = "faculty" THEN match ← projects where faculty = user
    ELSE IF user.role = "admin" THEN match ← all projects
    ELSE RETURN error 403
    apply the search text and the status and department filters
    sort by progress (highest or lowest first), deadline or name
    rows ← one page of results joined with team members and guide
    add live overdue counts for the projects on this page
    RETURN rows with team name, project title and progress percentage
```

# Chapter 7

# TESTING

Testing was done at three levels: automated tests of the backend API, automated tests of user-interface parts, and manual testing of the complete application in a web browser.

### Testing approach

- **API tests** were written with Jest and Supertest. They run against a real MongoDB server that mongodb-memory-server starts in memory, so database rules such as unique indexes are tested too. Each test starts with an empty database.
- **User-interface tests** were written with Vitest and Testing Library. They check the progress bar, status labels, error messages and the date and size formatting helpers.
- **Production build:** the Vite production build of the frontend must succeed. The backend runs directly on Node.js and needs no build step.
- **Manual testing** was done in the browser using the demo data, signing in as a student, a faculty member and an administrator, on a desktop screen and a phone-sized screen, in both light and dark themes.

**Table 7.1: Summary of automated tests**

| Area | Number of tests | Result |
|---|---|---|
| Sign-up, sign-in, sessions and password reset | 13 | All passed |
| Role-based access control | 8 | All passed |
| Teams | 6 | All passed |
| Proposals, reviews, submission and evaluation | 13 | All passed |
| Tasks and progress | 6 | All passed |
| Files and versions | 7 | All passed |
| Dashboards and public endpoints | 2 | All passed |
| **Total API tests** | **55** | **All passed** |
| User-interface components and helpers | 8 | All passed |
| Production build | — | Successful |

**Table 7.2: Selected automated test cases**

| ID | Test case | Expected result | Result |
|---|---|---|---|
| T01 | Sign up with an extra “role: admin” field | Account is created as a student; the role field is ignored | Pass |
| T02 | Sign in with a wrong password | Refused with a general message that does not say which part was wrong | Pass |
| T03 | A deactivated user uses an old token | Request is refused immediately | Pass |
| T04 | A student asks for the list of users | Refused (403 Forbidden) | Pass |
| T05 | A faculty member who is not the guide opens a project | Refused (403 Forbidden) | Pass |
| T06 | The guide rejects a proposal without feedback | Refused; feedback is required | Pass |
| T07 | A student tries to join a second team | Refused by the API and by the database index | Pass |
| T08 | A team tries to grow beyond its size limit | Refused with a clear message | Pass |
| T09 | A task is assigned to a student from another team | Refused | Pass |
| T10 | Tasks change status; one task passes its due date | Progress equals the share of done tasks; the late task is marked overdue | Pass |
| T11 | Upload a file of a blocked type or larger than 25 MB | Refused with a clear message | Pass |
| T12 | The guide uploads a new version without a comment | Refused; with a comment, version 2 is saved and version 1 is kept | Pass |
| T13 | A download link is tampered with | Refused | Pass |
| T14 | Submit the project without a final deliverable | Refused | Pass |
| T15 | Evaluate before submission, then after | Refused before; after, marks are saved with evaluator and date | Pass |
| T16 | Open the team progress board as each role | Admin sees all teams, faculty only their own, students are refused | Pass |

**Table 7.3: Manual test cases in the browser**

| ID | Test case | Expected result | Result |
|---|---|---|---|
| M01 | Student moves a task to Done on the board | Progress bar updates from 50% to 60% (10 tasks) | Pass |

| ID | Test case | Expected result | Result |
|---|---|---|---|
| M02 | Student uploads a file | File appears in the Files tab with uploader and time | Pass |
| M03 | Guide opens a text file | Contents are previewed inside the application | Pass |
| M04 | Guide uploads an edited version with a comment | Version 2 is shown, version 1 stays in the history with its comment | Pass |
| M05 | Admin opens the team progress page | All teams are listed with progress bars, highest first | Pass |
| M06 | Open the application on a 375-pixel-wide phone screen | No sideways scrolling; the menu opens as a drawer | Pass |
| M07 | Switch between light and dark themes | Both themes are readable on every page | Pass |
| M08 | Reload the page while signed in | The session is restored automatically | Pass |
| M09 | Run the production build served by the API | Pages load under the strict security policy with no errors | Pass |

**Results**

All 55 API tests and all 8 user-interface tests passed, the type check reported no errors and the production build succeeded. All manual test cases gave the expected result. Problems found during manual testing, such as a task due yesterday shown as “due today” and blank text-file previews, were fixed and checked again.

**Limitations of testing**

The Cloudinary, Amazon S3 and e-mail integrations are implemented and switched on through environment variables, but were not tested with live accounts, which need the institution's own credentials; files were stored on the server's disk during testing. Load testing has not been done yet.

# Chapter 8

## SCREENSHOTS

This chapter shows the main screens of EduTrack, captured from the running application with the demo data.

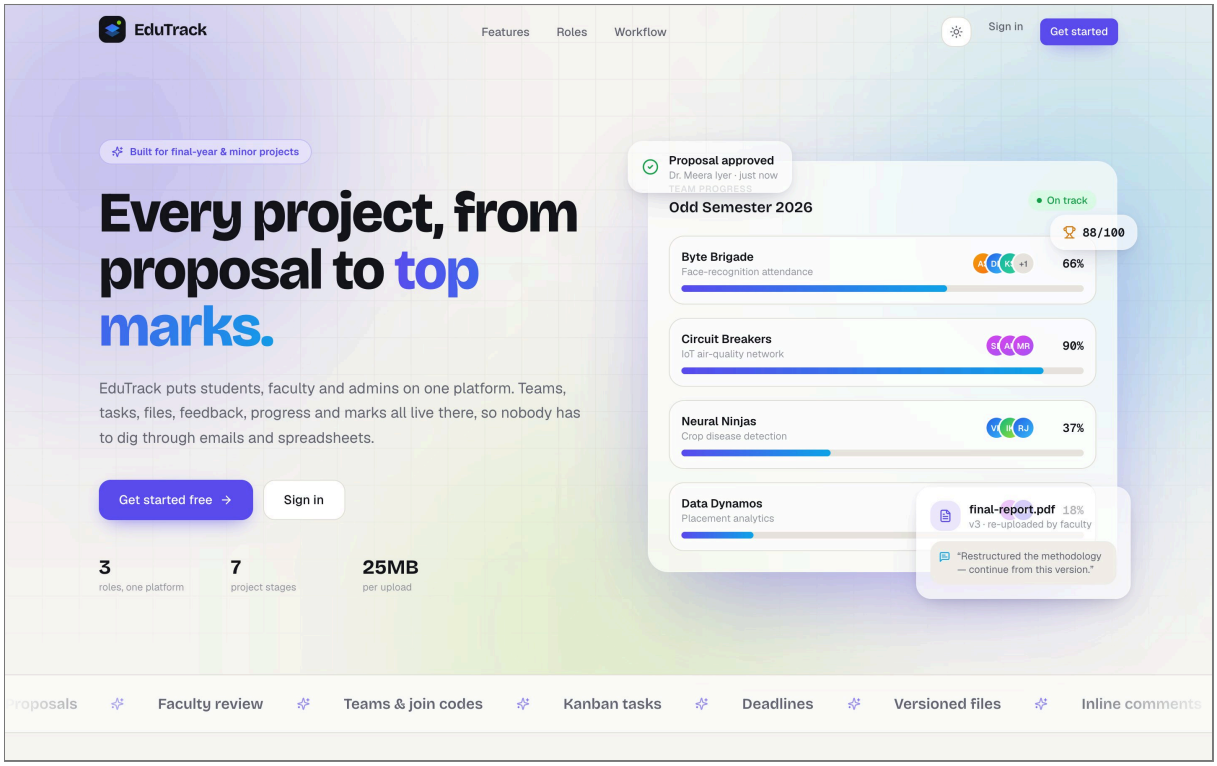



**Fig. 8.1: Home page of EduTrack**

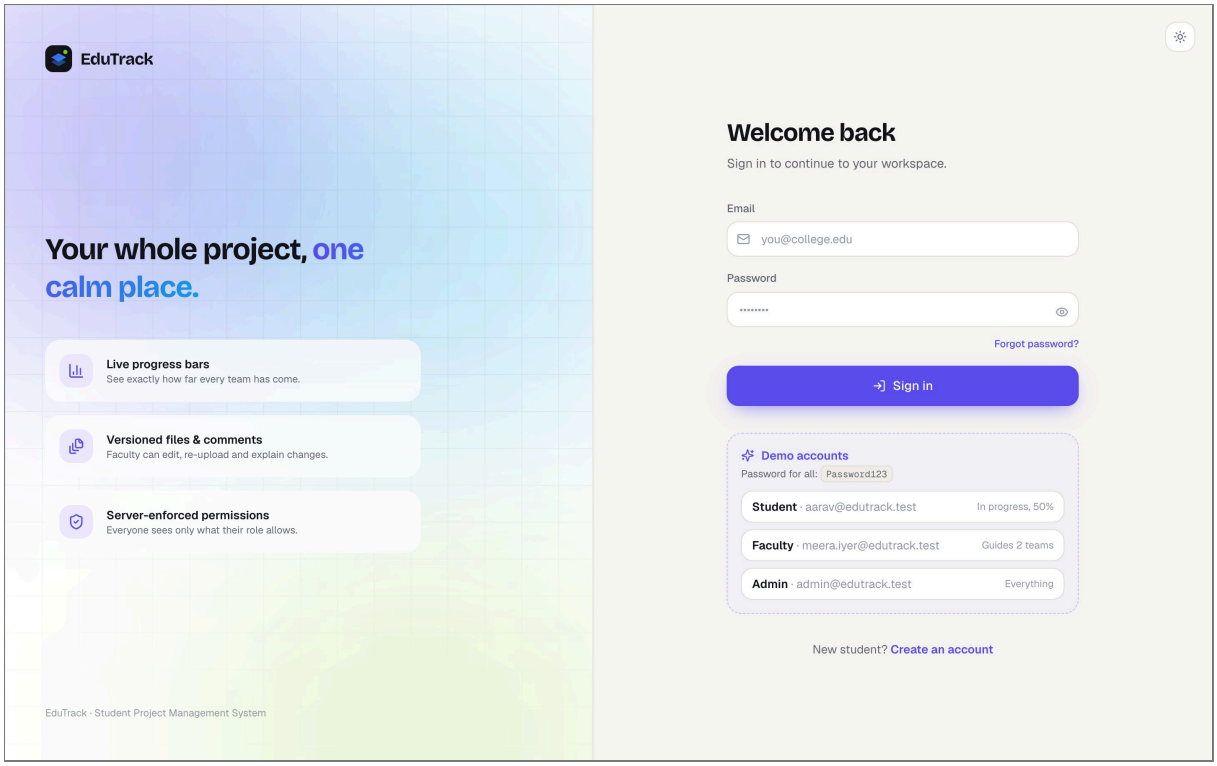



**Fig. 8.2: Sign-in page with demo accounts**

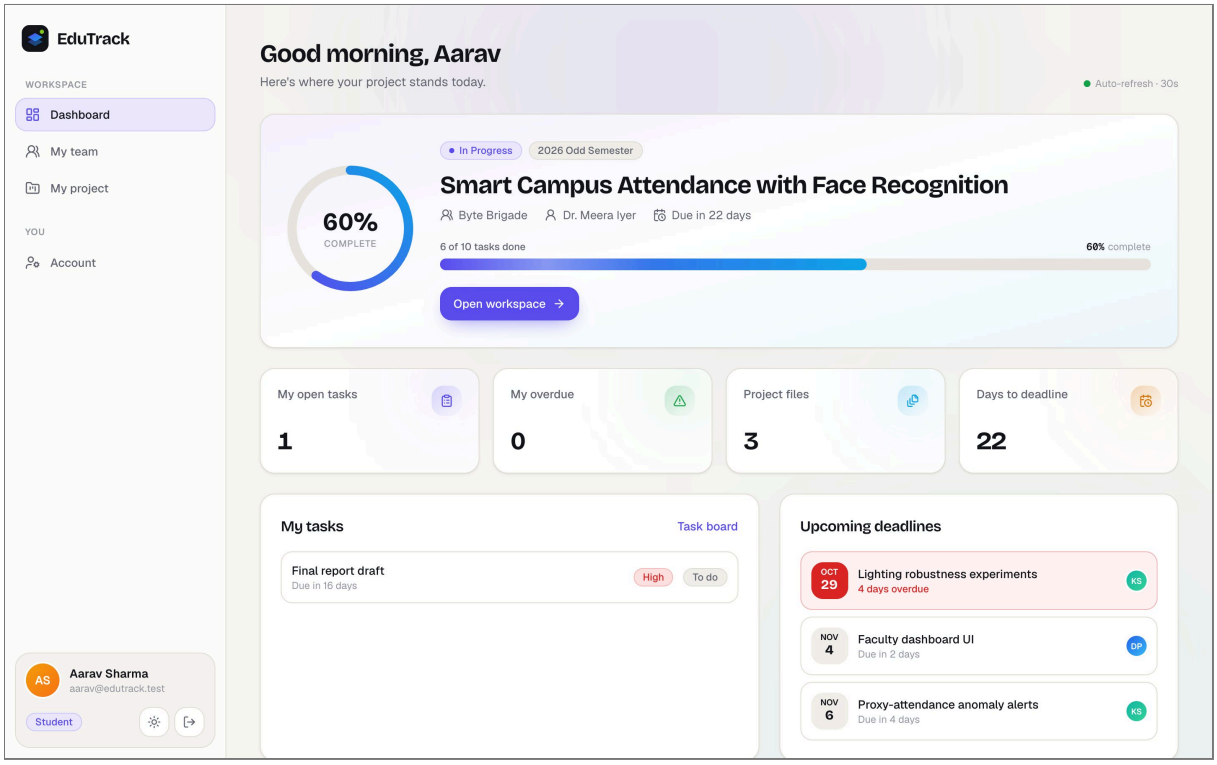



**Fig. 8.3: Student dashboard showing project progress, tasks and deadlines**

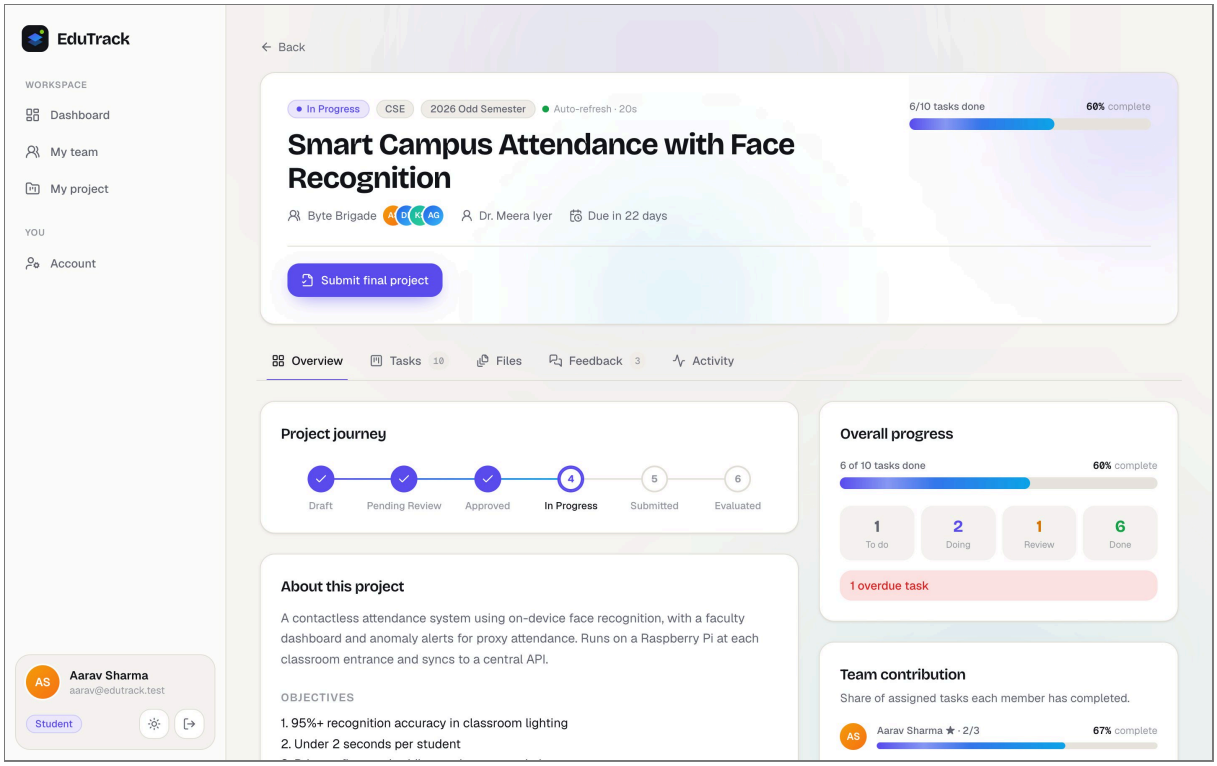



**Fig. 8.4: Project workspace: overview with the project journey**

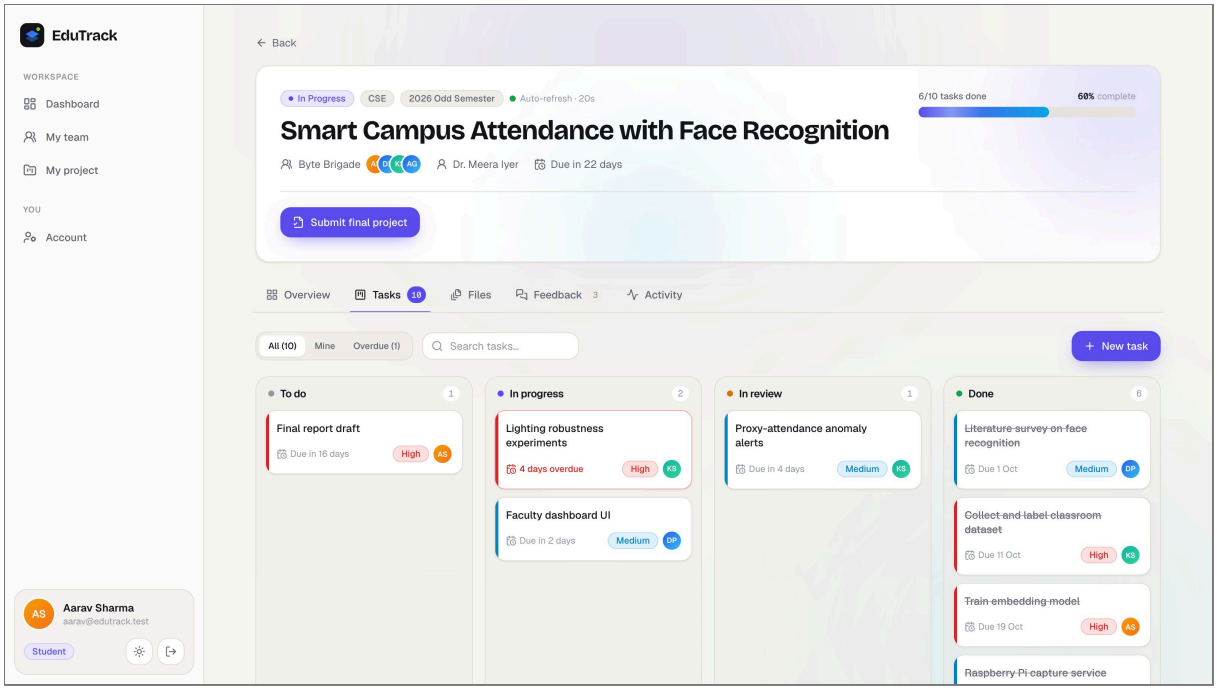



**Fig. 8.5: Task board with To do, In progress, In review and Done columns**

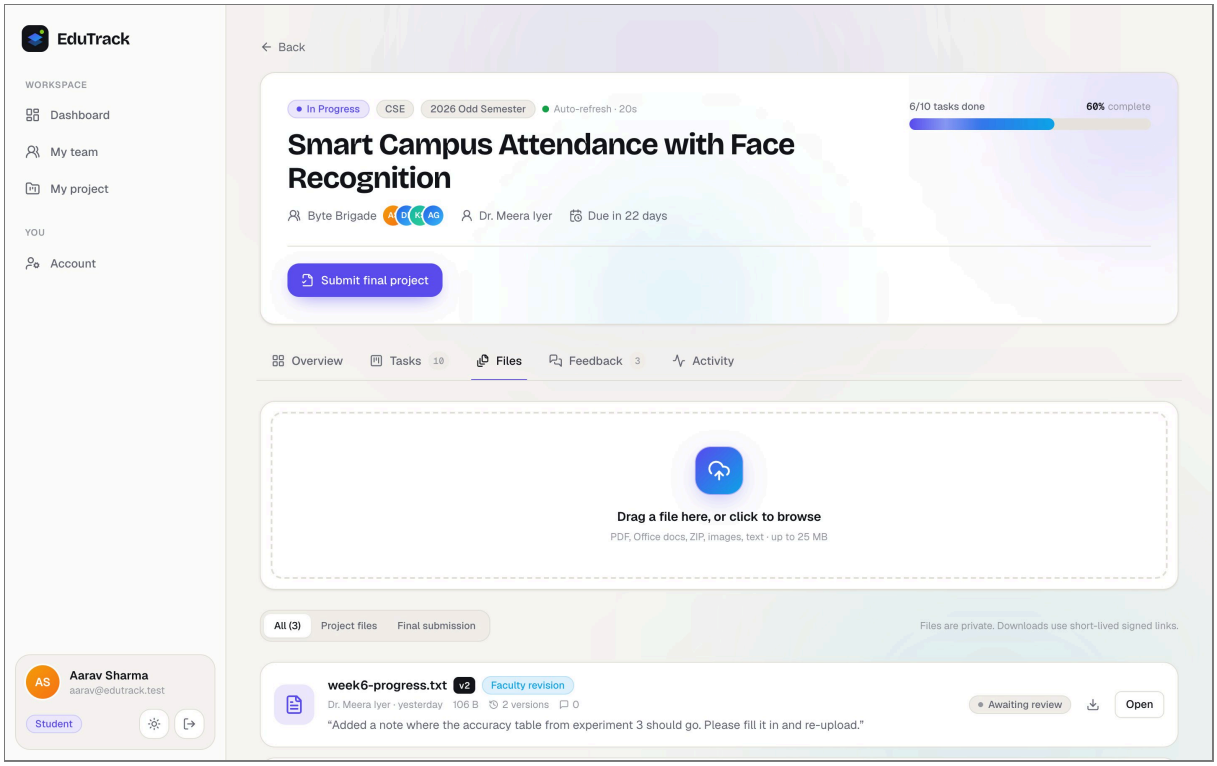



**Fig. 8.6: Files tab with the upload area and version labels**

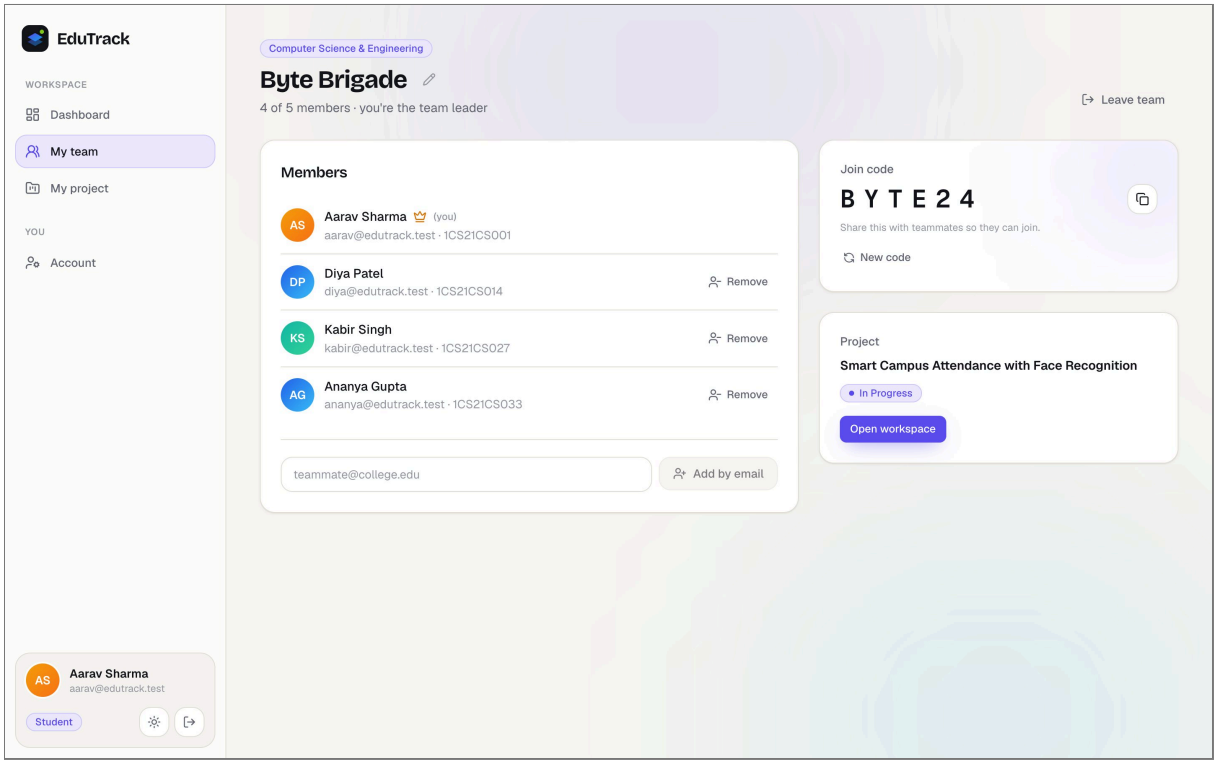



**Fig. 8.7: Team page with members and the join code**

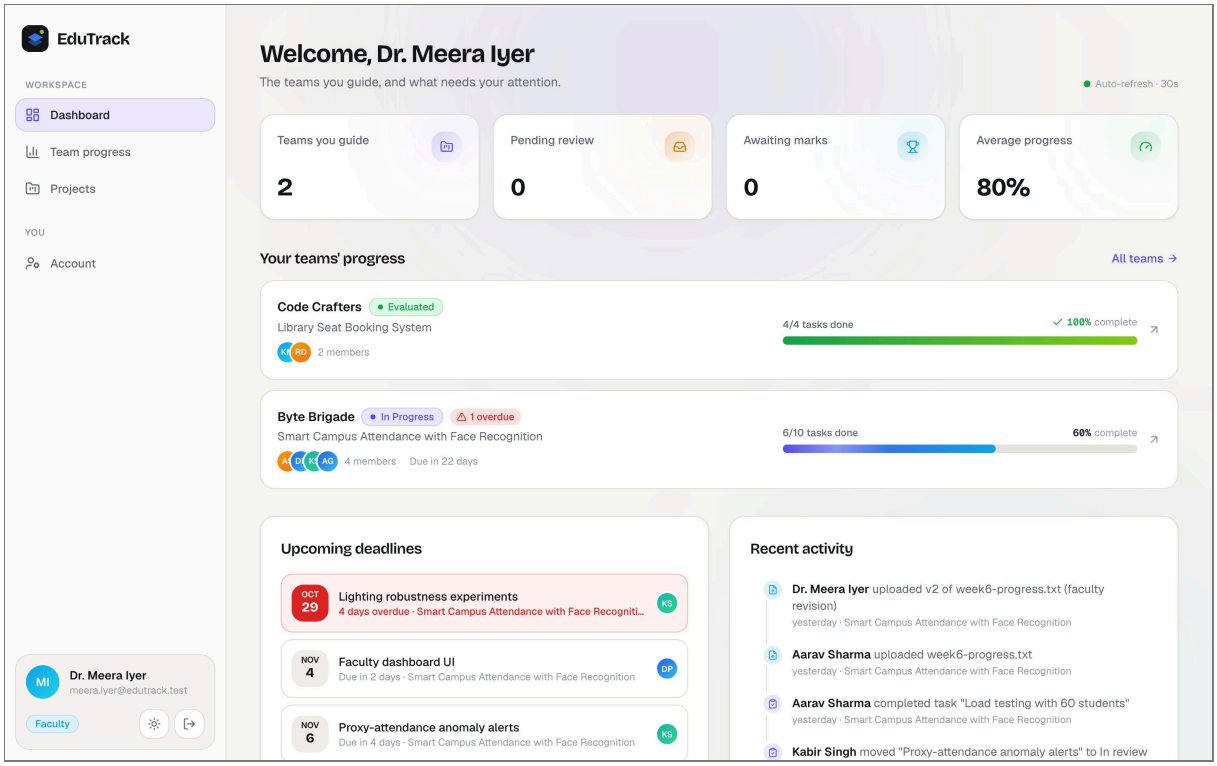



**Fig. 8.8: Faculty dashboard with progress bars of guided teams**

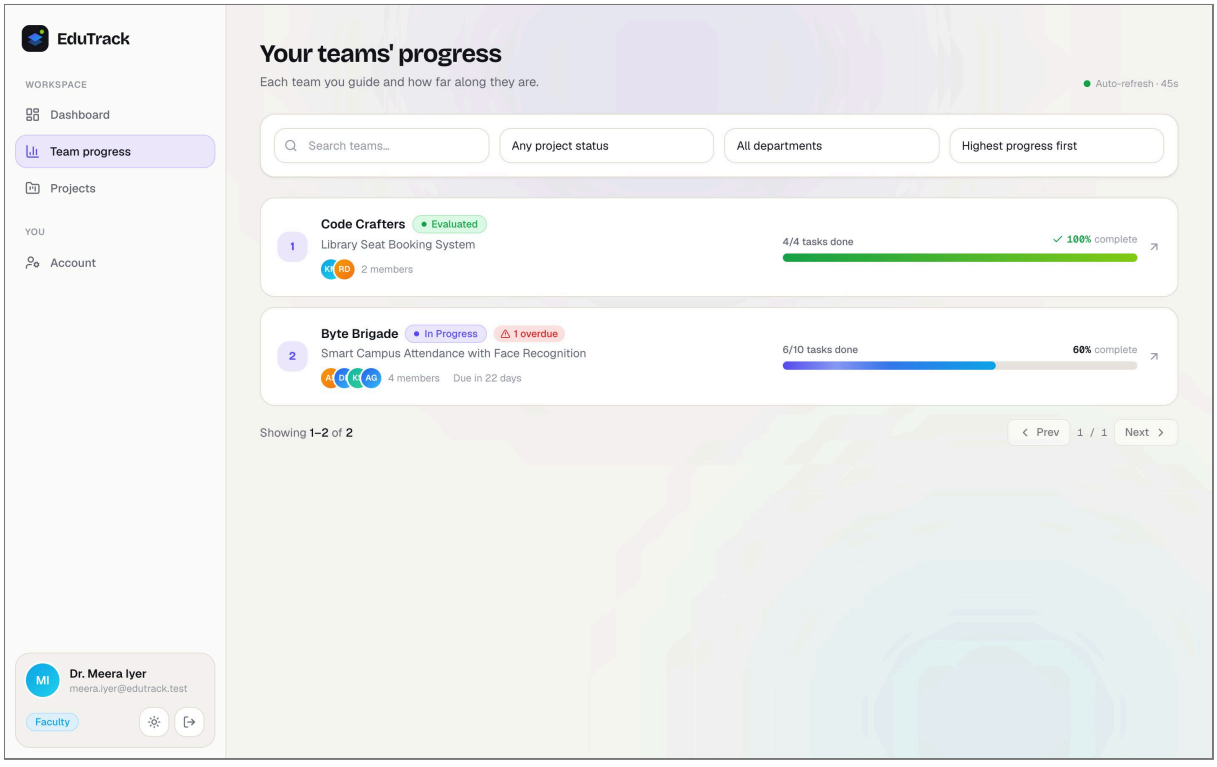



**Fig. 8.9: Faculty view of their teams sorted by progress**

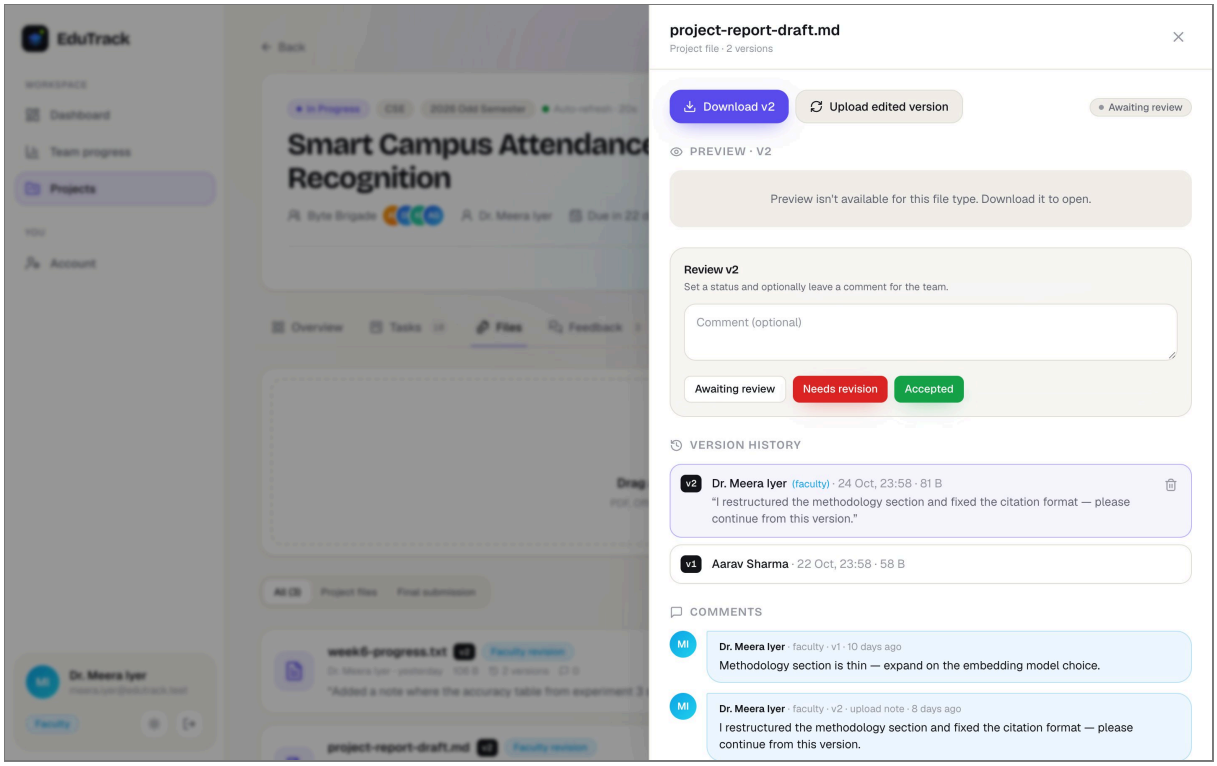



**Fig. 8.10: File details with review status, version history and comments**

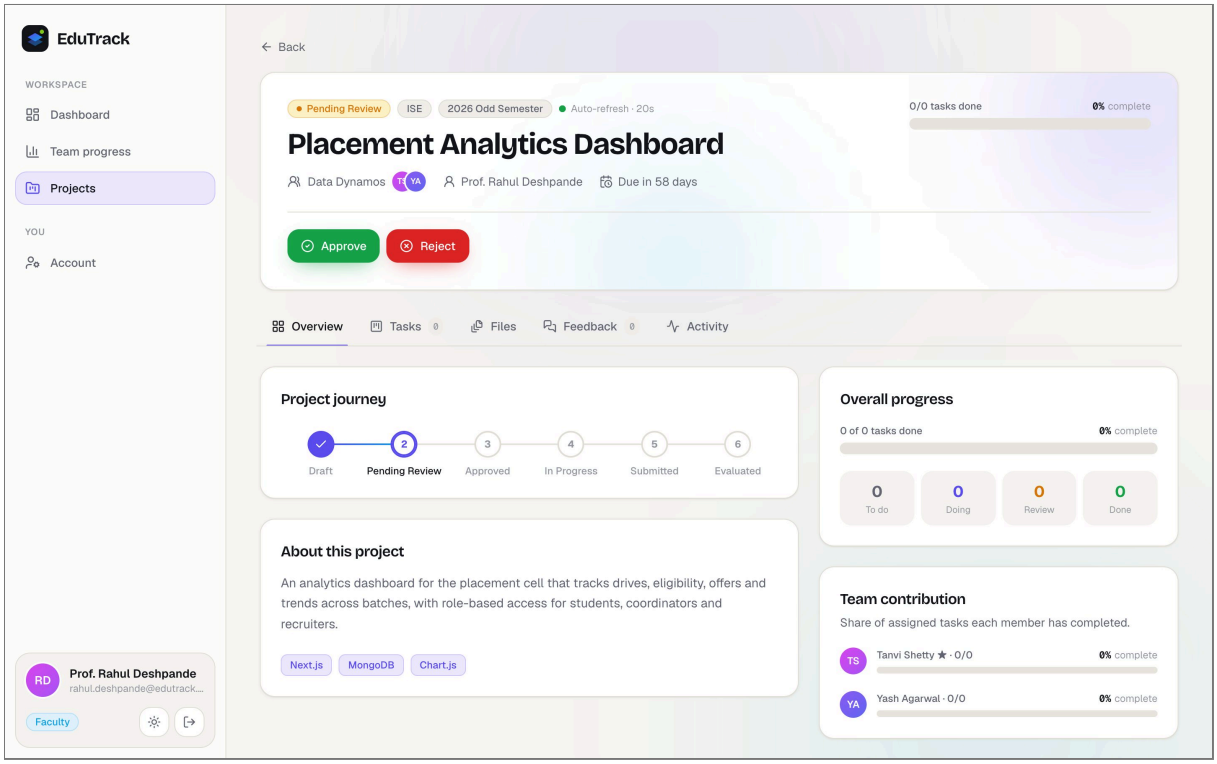



**Fig. 8.11: Proposal waiting for the guide's approval**

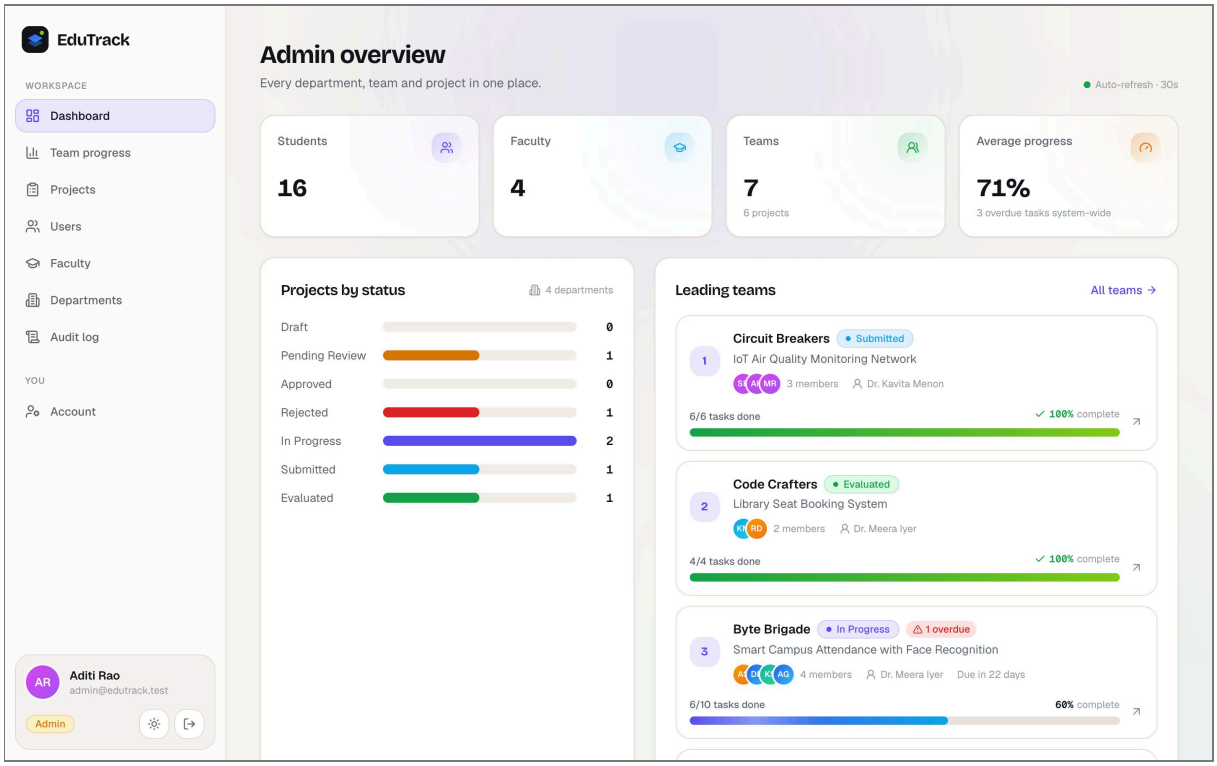



**Fig. 8.12: Administrator dashboard**

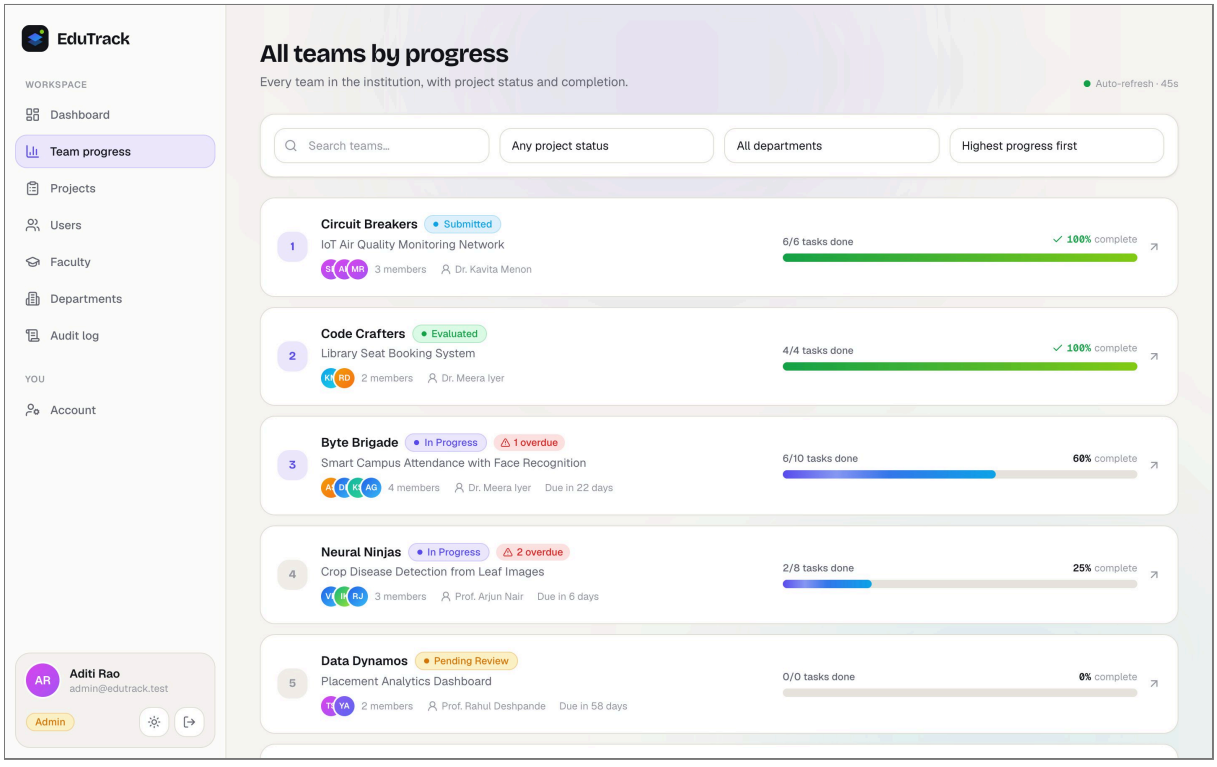



**Fig. 8.13: All teams in the institution ranked by progress**

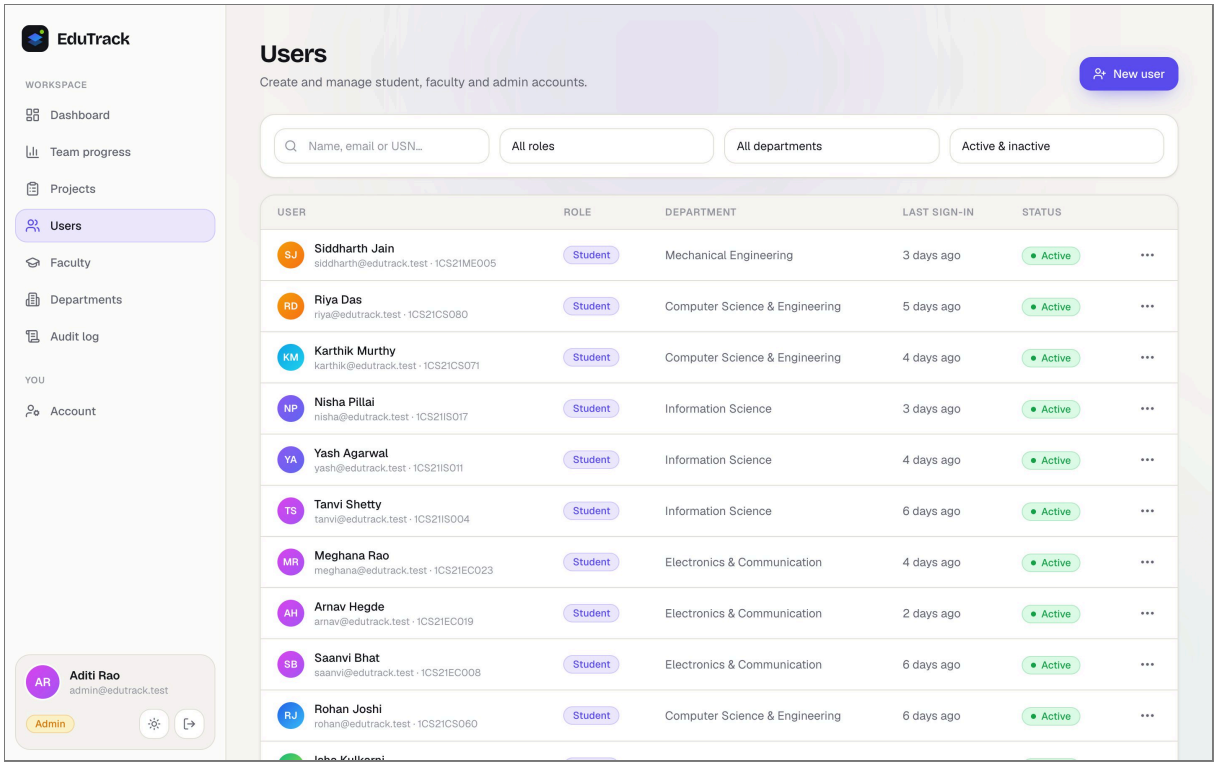



**Fig. 8.14: User management for the administrator**

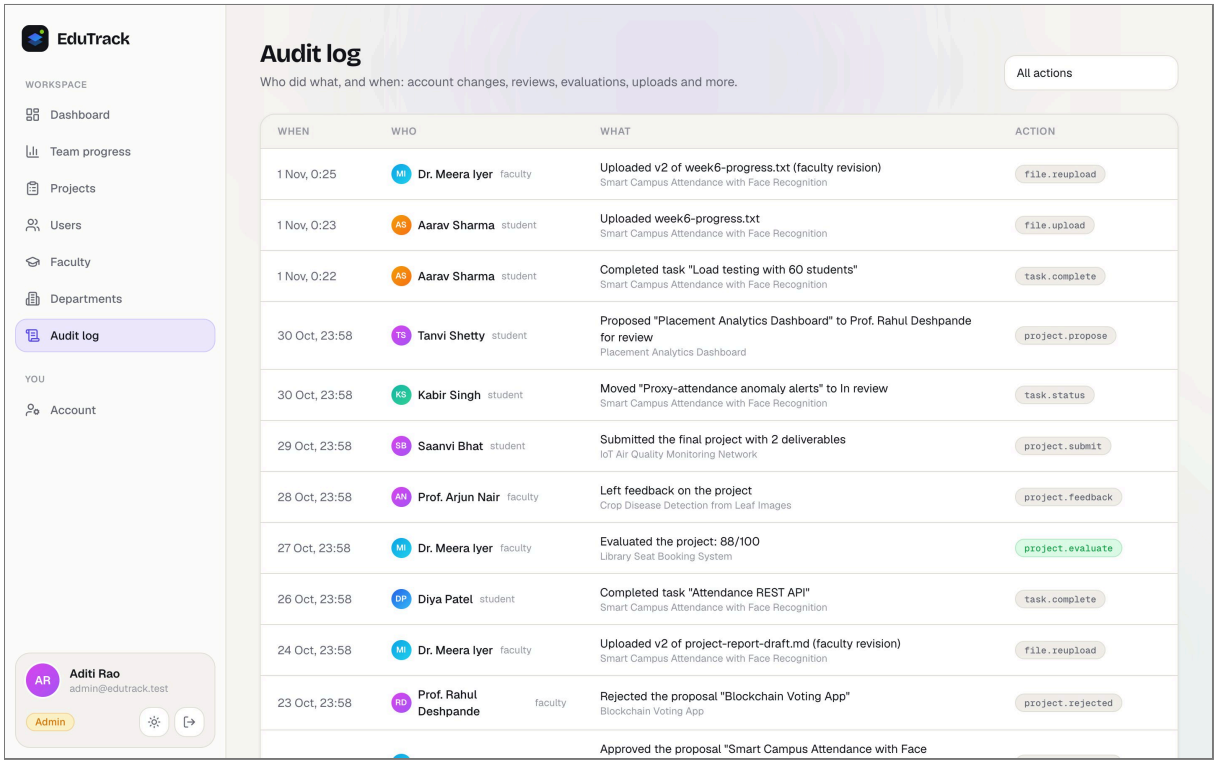



**Fig. 8.15: Audit log of important actions**

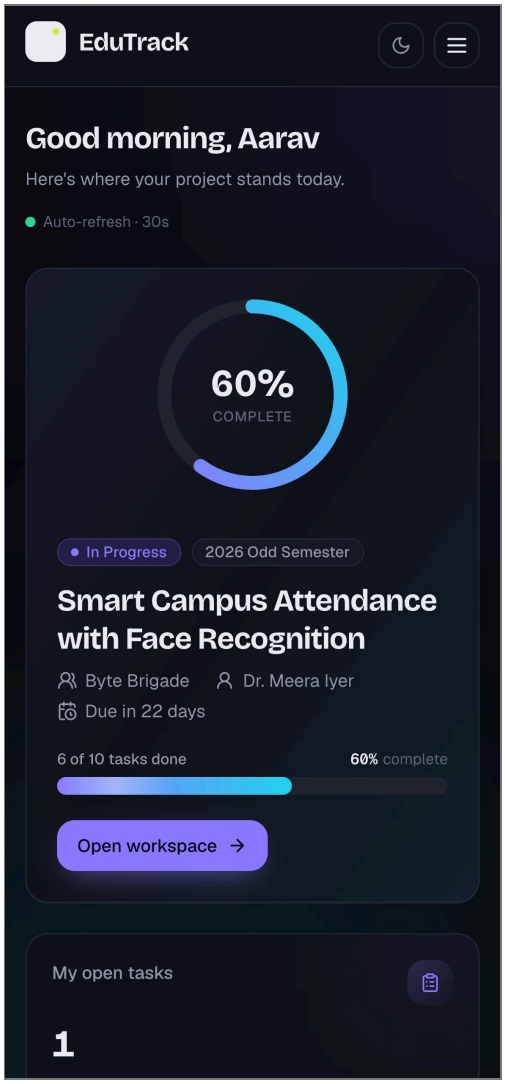



**Fig. 8.16: Student dashboard on a phone in the dark theme**

# FUTURE ENHANCEMENT AND CONCLUSION

**Future Enhancements**

- **Instant updates:** use WebSockets so that changes appear immediately instead of on the next automatic refresh.
- **Notifications:** e-mail and in-app alerts for approvals, new feedback and coming deadlines.
- **Calendar:** a calendar view of deadlines and export to Google Calendar.
- **College sign-in:** sign in with the college's Google or Microsoft accounts.
- **Detailed evaluation:** rubric-based marking by more than one examiner, with mark sheets exported as PDF or Excel files.
- **Plagiarism check:** connect to a plagiarism detection service for reports.
- **Analytics:** charts of progress over time and of faculty workload for the department.
- **Mobile app:** an installable app with offline viewing of project details.

**Conclusion**

EduTrack replaces the scattered mix of chats, e-mails, pen drives and spreadsheets with one web application for managing academic projects. Students form teams, propose their project, plan the work as tasks and share files. Guides review proposals, correct files with comments kept in the version history, follow every team's progress and award marks. Administrators manage users, departments and projects and see all teams ranked by progress.

The project met the objectives set in the synopsis. Progress is calculated automatically and shown as a clear percentage to every role, permissions are enforced on the server for every request, and every file version is preserved. The system is covered by 55 automated API tests and 8 user-interface tests, and is packaged with Docker and Render configuration for deployment with the institution's own database and storage accounts.

Working on this project gave us practical experience of the full process of building web software: understanding the needs of different users, designing a database, building a secure API and a responsive interface, writing automated tests and preparing an application for deployment.

# REFERENCES


[1] Moodle Pty Ltd, "Moodle documentation," https://docs.moodle.org/ (accessed Oct. 2026).

[2] Google, "Google Classroom Help," https://support.google.com/edu/classroom/ (accessed Oct. 2026).

[3] Atlassian, "Trello guide," https://trello.com/guide (accessed Oct. 2026).

[4] Atlassian, "Jira Software Cloud documentation," https://support.atlassian.com/jira-software-cloud/ (accessed Oct. 2026).

[5] GitHub, "GitHub Classroom documentation," https://docs.github.com/en/education/manage-coursework-with-github-classroom (accessed Oct. 2026).

[6] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Sequim, WA, USA: Blue Hole Press, 2010.

[7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[8] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[9] OpenJS Foundation, "Express: Node.js web application framework," https://expressjs.com/ (accessed Oct. 2026).

[10] Meta Open Source, "React documentation," https://react.dev/ (accessed Oct. 2026).

[11] MongoDB, Inc., "MongoDB manual," https://www.mongodb.com/docs/manual/ (accessed Oct. 2026).

[12] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[13] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91.

[14] OWASP Foundation, "OWASP Top 10: 2021," https://owasp.org/Top10/ (accessed Oct. 2026).

[15] Mongoose, "Mongoose ODM documentation," https://mongoosejs.com/docs/ (accessed Oct. 2026).

[16] TanStack, "TanStack Query documentation," https://tanstack.com/query/latest (accessed Oct. 2026).

[17] Zod, "Zod documentation," https://zod.dev/ (accessed Oct. 2026).

[18] A. Barth, "HTTP state management mechanism," RFC 6265, Internet Engineering Task Force, Apr. 2011.